



OPTIMIZACIÓN

PROBLEMA DE LA MOCHILA (KNAPSACK PROBLEM)

Josefa Iturra González
Rafael Rodríguez Tovar

PROBLEMA

El problema de la mochila consiste en seleccionar un conjunto de objetos, cada uno con un peso y un valor, de manera que no se supere la capacidad máxima de la mochila y se logre maximizar la utilidad total.

EJEMPLO ILUSTRATIVO

Tenemos un viaje súper importante a una convención urbana donde queremos vender algunos de nuestros objetos, sin embargo, como equipaje, sólo podemos llevar una mochila que posee una capacidad máxima de 6 kg.

Objetos:

- Laptop: 2 kg / \$10
- PlayStation 4: 0.5 kg / \$6
- Jordan Retro 4: 1 kg / \$5
- Balón de básquet: 3 kg / \$3
- Gorra New Era: 0.75 kg / \$2

MODELO MATEMATICO

VARIABLES DE DECISIÓN

$$x_i \in \{0, 1\}$$

indica si el objeto es
seleccionado (1) o no es
seleccionado (0).

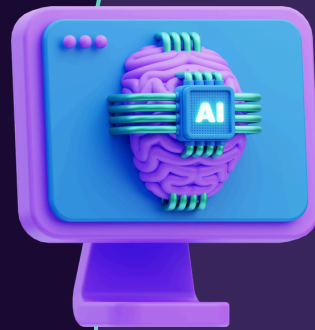
RESTRICCIONES PRINCIPALES

$$\sum_{i=1}^n w_i x_i \leq C$$

FUNCIÓN OBJETIVO

$$\sum_{i=1}^n v_i x_i$$

COMPLEJIDAD



NP-Completo

NP es una clase de problemas cuyas soluciones pueden verificarse en tiempo polinomial.

Es decir, es fácil comprobar si una solución es correcta, aunque no necesariamente encontrarla.



NP-HARD

Un problema es NP-Hard cuando es tan difícil como los problemas más complejos de NP.

Otros problemas, como Subset Sum, pueden transformarse en este en tiempo polinomial.

IMPLEMENTACIÓN

PROGRAMACIÓN DINAMICA CON MEMORIZACIÓN

La programación dinámica divide el problema en subproblemas más pequeños, resolviendo cada uno para construir la solución óptima global.

La memoización permite almacenar resultados ya calculados, evitando recomputarlos y reduciendo significativamente el tiempo de ejecución.

BACKTRACKING

El backtracking explora todas las posibles combinaciones de solución mediante recursión, descartando aquellas que no cumplen con las restricciones.

En el problema de la mochila, se realiza poda cuando el peso acumulado supera la capacidad, evitando seguir explorando soluciones inválidas.

RESULTADOS EXPERIMENTALES

TENDENCIA DE TIEMPOS DE EJECUCIÓN

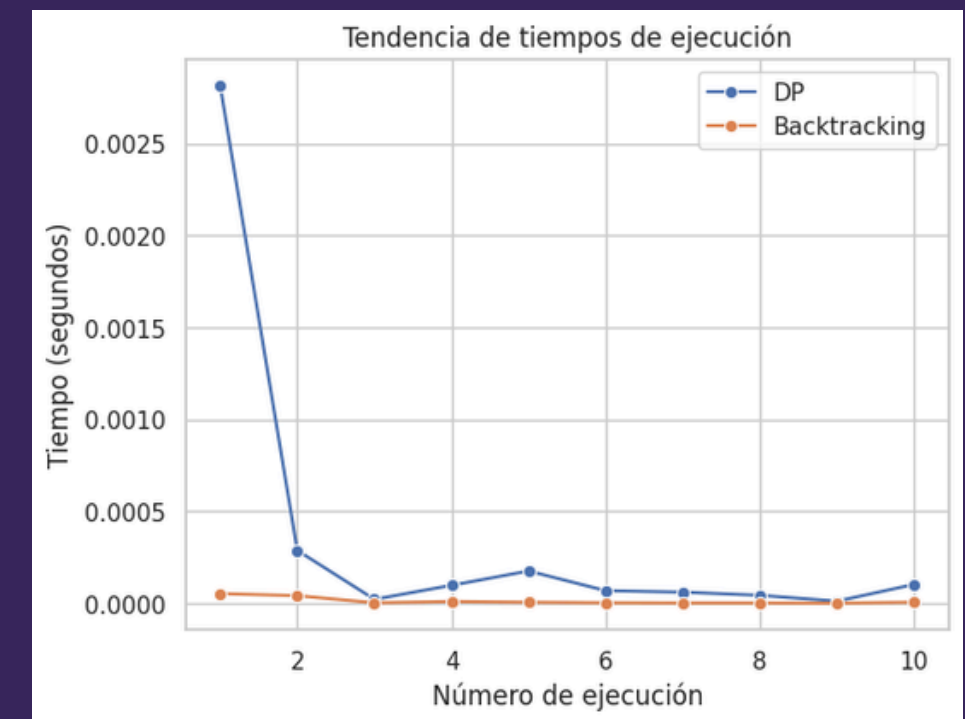
PYTHON



C++



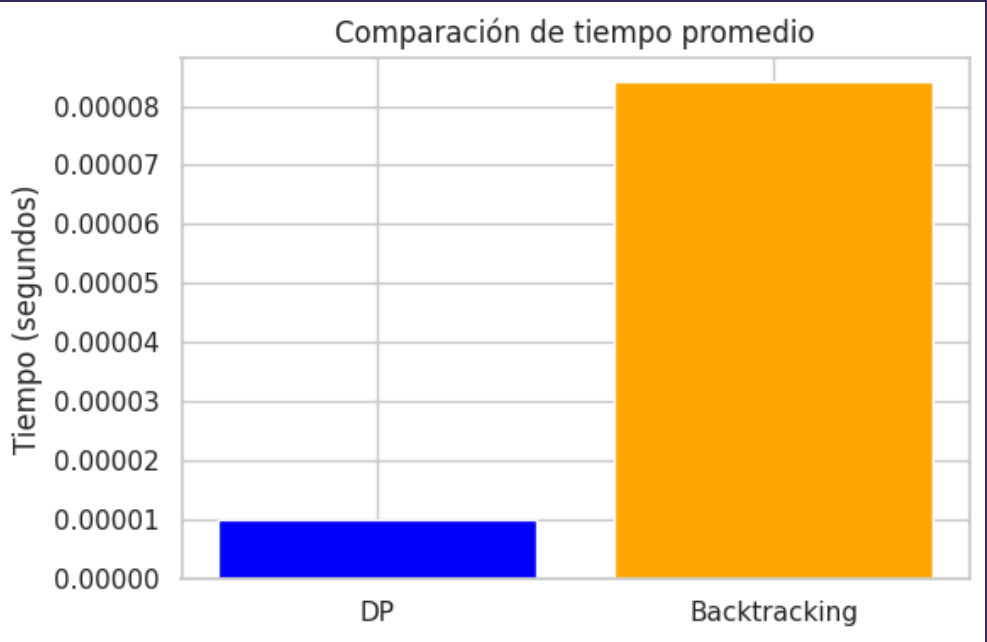
JAVA



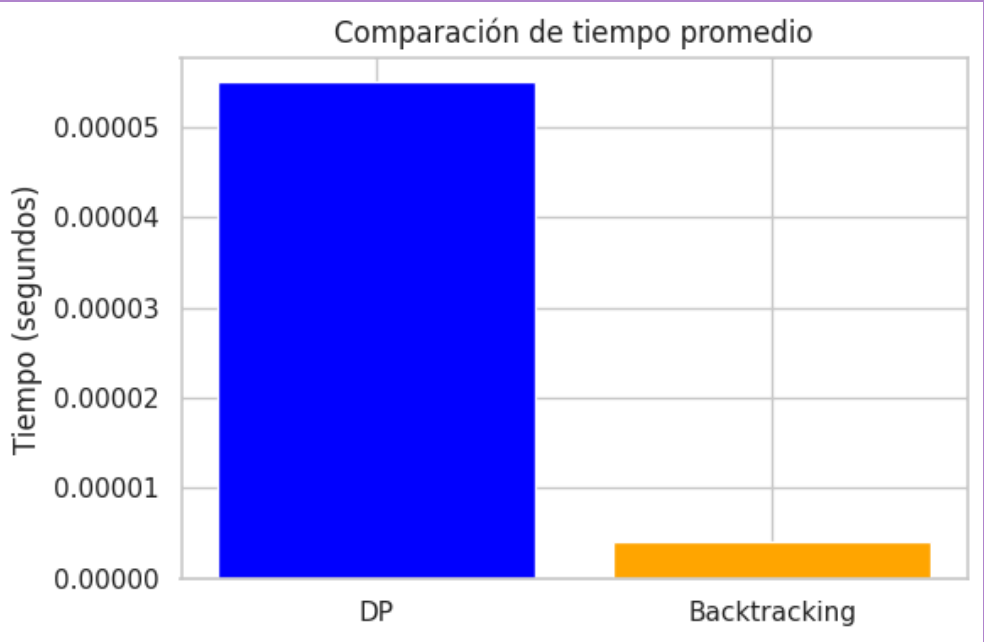
RESULTADOS EXPERIMENTALES

TIEMPO PROMEDIO DE EJECUCIÓN

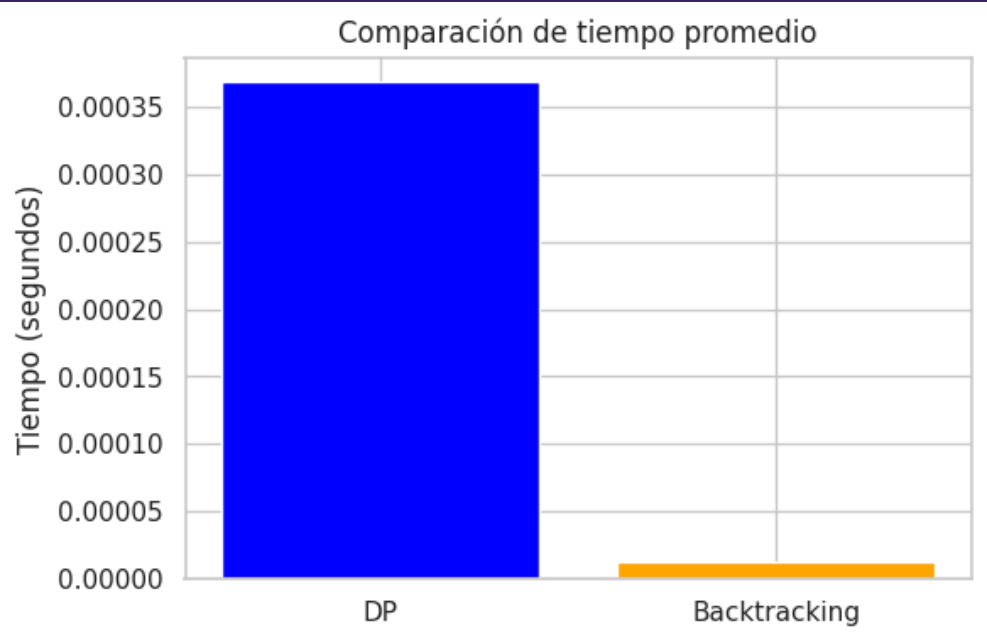
PYTHON



C++



JAVA



RESULTADOS EXPERIMENTALES

TABLAS DE RESULTADOS

PYTHON

...	Ejemplo	n	Capacidad	Tiempo_DP	Resultado_DP	Tiempo_BT	Resultado_BT
0	1	9	25	0.000029	72	0.000212	85
1	2	9	26	0.000002	89	0.000159	94
2	3	7	13	0.000009	36	0.000009	32
3	4	10	12	0.000024	37	0.000045	50
4	5	9	30	0.000018	74	0.000128	68
5	6	8	27	0.000004	61	0.000073	65
6	7	7	28	0.000010	67	0.000043	80
7	8	6	26	0.000001	53	0.000021	65
8	9	5	11	0.000001	28	0.000010	35
9	10	9	26	0.000001	89	0.000140	89

C++

```
=== DP ===
Ejemplo 1 | n=9 | C=25 | Tiempo: 0.000096 s | Resultado: 85
Ejemplo 2 | n=9 | C=26 | Tiempo: 0.000088 s | Resultado: 94
Ejemplo 3 | n=7 | C=13 | Tiempo: 0.000007 s | Resultado: 32
Ejemplo 4 | n=10 | C=12 | Tiempo: 0.000035 s | Resultado: 50
Ejemplo 5 | n=9 | C=30 | Tiempo: 0.000094 s | Resultado: 68
Ejemplo 6 | n=8 | C=27 | Tiempo: 0.000055 s | Resultado: 65
Ejemplo 7 | n=7 | C=28 | Tiempo: 0.000054 s | Resultado: 80
Ejemplo 8 | n=6 | C=26 | Tiempo: 0.000025 s | Resultado: 65
Ejemplo 9 | n=5 | C=11 | Tiempo: 0.000010 s | Resultado: 35
Ejemplo 10 | n=9 | C=26 | Tiempo: 0.000085 s | Resultado: 89

=== Backtracking ===
Ejemplo 1 | n=9 | C=25 | Tiempo: 0.000007 s | Resultado: 85
Ejemplo 2 | n=9 | C=26 | Tiempo: 0.000008 s | Resultado: 94
Ejemplo 3 | n=7 | C=13 | Tiempo: 0.000001 s | Resultado: 32
Ejemplo 4 | n=10 | C=12 | Tiempo: 0.000003 s | Resultado: 50
Ejemplo 5 | n=9 | C=30 | Tiempo: 0.000007 s | Resultado: 68
Ejemplo 6 | n=8 | C=27 | Tiempo: 0.000004 s | Resultado: 65
Ejemplo 7 | n=7 | C=28 | Tiempo: 0.000002 s | Resultado: 80
Ejemplo 8 | n=6 | C=26 | Tiempo: 0.000001 s | Resultado: 65
Ejemplo 9 | n=5 | C=11 | Tiempo: 0.000001 s | Resultado: 35
Ejemplo 10 | n=9 | C=26 | Tiempo: 0.000007 s | Resultado: 89
```

JAVA

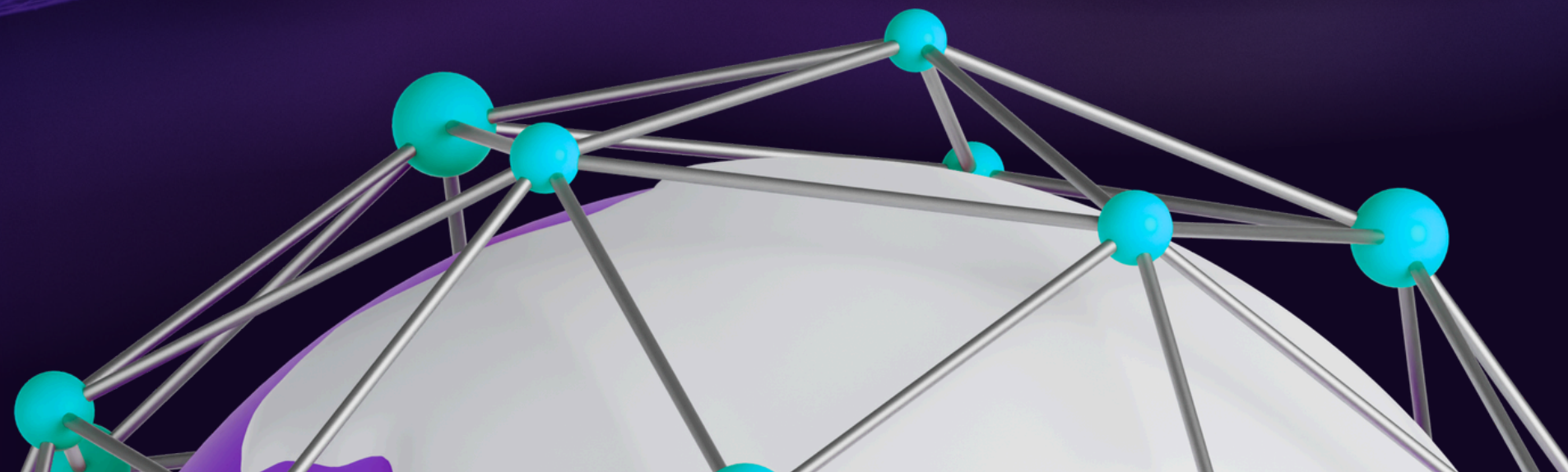
```
=== DP ===
Ejemplo 1 | n=9 | C=25 | Tiempo: 0.002815 s | Resultado: 85
Ejemplo 2 | n=9 | C=26 | Tiempo: 0.000288 s | Resultado: 94
Ejemplo 3 | n=7 | C=13 | Tiempo: 0.000022 s | Resultado: 32
Ejemplo 4 | n=10 | C=12 | Tiempo: 0.000098 s | Resultado: 50
Ejemplo 5 | n=9 | C=30 | Tiempo: 0.000176 s | Resultado: 68
Ejemplo 6 | n=8 | C=27 | Tiempo: 0.000069 s | Resultado: 65
Ejemplo 7 | n=7 | C=28 | Tiempo: 0.000061 s | Resultado: 80
Ejemplo 8 | n=6 | C=26 | Tiempo: 0.000044 s | Resultado: 65
Ejemplo 9 | n=5 | C=11 | Tiempo: 0.000012 s | Resultado: 35
Ejemplo 10 | n=9 | C=26 | Tiempo: 0.000103 s | Resultado: 89

=== Backtracking ===
Ejemplo 1 | n=9 | C=25 | Tiempo: 0.000053 s | Resultado: 85
Ejemplo 2 | n=9 | C=26 | Tiempo: 0.000042 s | Resultado: 94
Ejemplo 3 | n=7 | C=13 | Tiempo: 0.000003 s | Resultado: 32
Ejemplo 4 | n=10 | C=12 | Tiempo: 0.000010 s | Resultado: 50
Ejemplo 5 | n=9 | C=30 | Tiempo: 0.000006 s | Resultado: 68
Ejemplo 6 | n=8 | C=27 | Tiempo: 0.000003 s | Resultado: 65
Ejemplo 7 | n=7 | C=28 | Tiempo: 0.000002 s | Resultado: 80
Ejemplo 8 | n=6 | C=26 | Tiempo: 0.000002 s | Resultado: 65
Ejemplo 9 | n=5 | C=11 | Tiempo: 0.000001 s | Resultado: 35
Ejemplo 10 | n=9 | C=26 | Tiempo: 0.000006 s | Resultado: 89
```

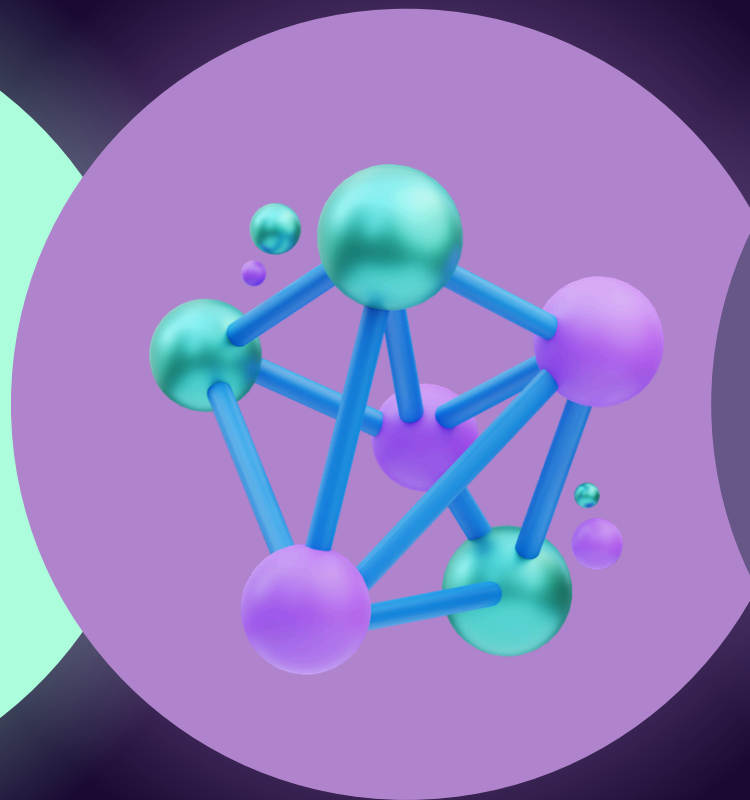
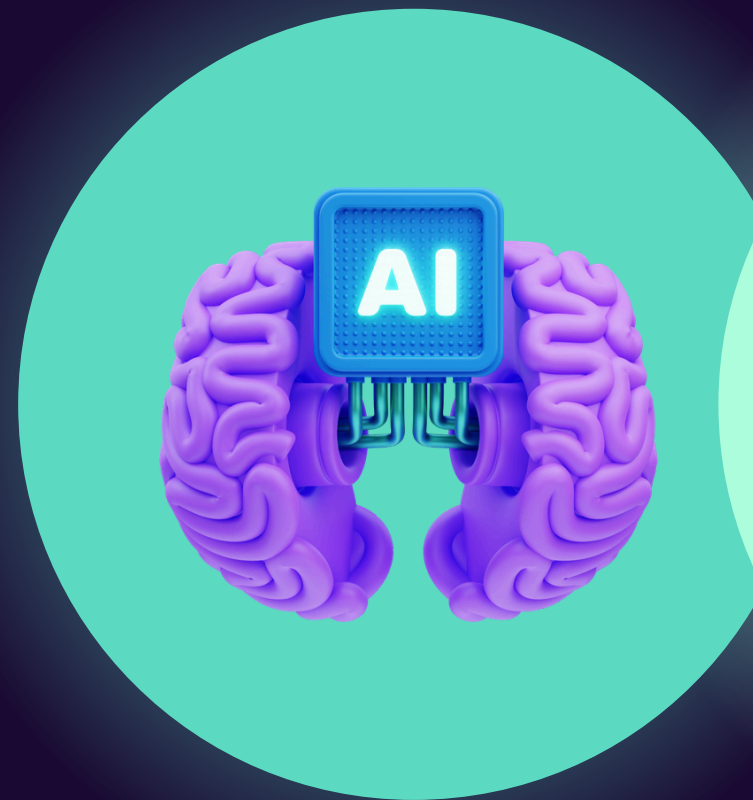
ESCALABILIDAD

La programación dinámica presenta mejor escalabilidad que el backtracking, ya que evita recomputaciones mediante el uso de memoria.

El backtracking crece exponencialmente al explorar todas las combinaciones, por lo que se vuelve ineficiente en instancias grandes.



CONCLUSIONES



The background is a deep purple color. In the center, there is a glowing, three-dimensional wireframe grid that forms a wavy, undulating shape, resembling a landscape or a stylized brain. On the left and right sides, there are partial, 3D-rendered illustrations of human brains, colored in a lighter shade of purple. The text is centered over the wireframe shape.

**¡GRACIAS POR
LA ATENCIÓN!**